



Частное профессиональное образовательное учреждение
«КОЛЛЕДЖ СОВРЕМЕННЫХ ТЕХНОЛОГИЙ И МЕДИЦИНЫ»

Междисциплинарная аттестационная работа.

на тему «ИС «Кинотеатр» (Залы, Фильмы, Сеансы, Билеты).»

формы обучения **очная**

подчеркнуть нужное

специальность 09.02.07 Информационные системы и программирование
(код и наименование)

Обучающийся
группы

Е.Э.Афониная
К.О.Козина
Э.И.Штейн

подпись, дата

И.О. Фамилия

инициалы и фамилия

Оценка выполненной обучающимся работы: _____

Преподаватель,
уч. степ., уч. зв.

подпись, дата

И.О. Фамилия

инициалы и фамилия

Москва, 2026г

Роли исполнителей проекта:

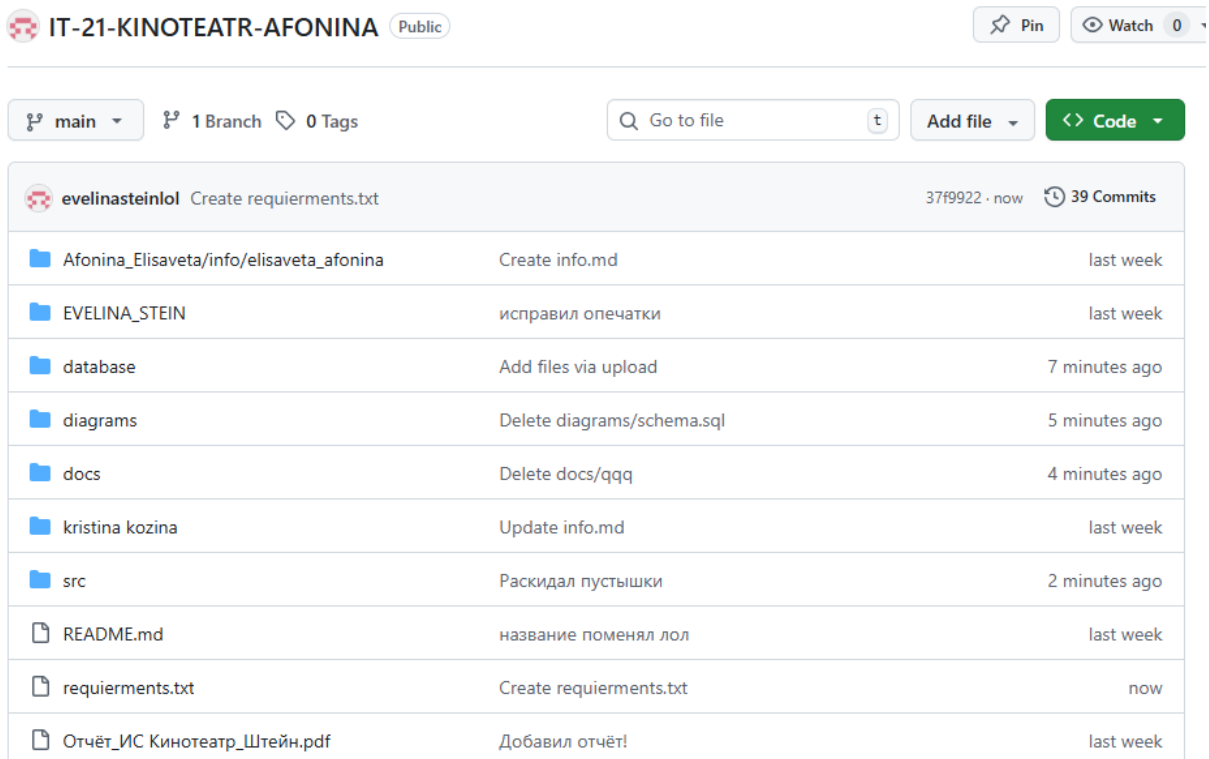
Елизавета Эдуардовна Афонина - Тимлид + Frontend-разработчик

Кристина Олеговна Козина - Backend-разработчик

Эвелина Игоревна Штейн - Аналитик + Алгоритмист

Цель работы: научиться подключать Python к SQLite; запускать БД из *schema.sql*; писать Python-код для работы с БД; реализовывать CRUD для своих сущностей проекта; делать отчёты и предметную логику для своего проекта; оформлять всё как часть командного проекта.

Подготовка структуры проекта.



IT-21-KINOTEATR-AFONINA Public		Pin	Watch 0
main	1 Branch	0 Tags	Go to file
Add file		Code	
evelinasteinlol Create requierments.txt 37f9922 · now 39 Commits			
Afonina_Elisaveta/info/elisaveta_afonina	Create info.md	last week	
EVELINA_STEIN	исправил опечатки	last week	
database	Add files via upload	7 minutes ago	
diagrams	Delete diagrams/schema.sql	5 minutes ago	
docs	Delete docs/qqq	4 minutes ago	
kristina kozina	Update info.md	last week	
src	Раскидал пустышки	2 minutes ago	
README.md	название поменял лол	last week	
requierments.txt	Create requierments.txt	now	
Отчёт_ИС Кинотеатр_Штейн.pdf	Добавил отчёт!	last week	

На платформе GitHub, где лежит вся наша работа, мы создали папки по примеру из задания. А именно – database, scr и docs!.

Их содержимое чуть ниже:

IT-21-KINOTEATR-AFONINA / database / 



evelinasteinlol Add files via upload

Name
 ..
 app.db
 schema.sql

IT-21-KINOTEATR-AFONINA / docs / 



evelinasteinlol Delete docs/qqq

Name
 ..
 ER.png

IT-21-KINOTEATR-AFONINA / src /



evelinasteinlol Раскидал пустышки

Name



..



db.py



main.py



reports.py



services.py

Создание файла requierments.txt



README.md



requierments.txt

На данный момент файл пуст

Содержание файлов

src/db.py

Code

Blame

32 lines (26 loc) · 1.15 KB



```
1  import os
2  import sqlite3
3
4  # Настройка путей
5  BASE_DIR = os.path.dirname(os.path.abspath(__file__))
6
7  # Путь к файлу базы данных
8  DB_PATH = os.path.join(BASE_DIR, 'database', 'app.db')
9
10 # Путь к схеме (schema.sql)
11 SCHEMA_PATH = os.path.join(BASE_DIR, 'schema.sql')
12
13 def get_connection():
14     """Создает подключение с поддержкой имен колонок и связей."""
15     conn = sqlite3.connect(DB_PATH)
16     conn.row_factory = sqlite3.Row
17     conn.execute("PRAGMA foreign_keys = ON;")
18     return conn
19
20 def init_db():
21     """Инициализация базы из schema.sql."""
22     if not os.path.exists(SCHEMA_PATH):
23         print(f"Ошибка! Файл {SCHEMA_PATH} не найден. Проверь папку проекта.")
24         return
25
26     try:
27         with get_connection() as conn:
28             with open(SCHEMA_PATH, 'r', encoding='utf-8') as f:
29                 conn.executescript(f.read())
30             print("--- Структура базы данных успешно создана и заполнена ---")
31     except sqlite3.Error as e:
32         print(f"Ошибка SQLite: {e}")
```

src/main.py

Code

Blame

78 lines (66 loc) · 3.35 KB



```
1 import sys
2 from db import init_db
3 from services import add_movie, add_session, add_department, add_employee
4 from reports import show_all_movies # Импортируем функцию отчета
5
6 def main():
7     print("=== Система управления кинотеатром v1.0 ===")
8     # Инициализация базы данных (если требуется)
9     init_db()
10
11     while True:
12         print("\nВыберите действие:")
13         print("1. Показать все фильмы (Отчет)")
14         print("2. Добавить новый фильм")
15         print("3. Добавить сеанс")
16         print("4. Добавить отдел")
17         print("5. Добавить сотрудника")
18         print("0. Выход")
19
20         choice = input("\nВведите номер: ")
21
22         if choice == "1":
23             # вызываем функцию отображения фильмов
24             show_all_movies()
25
26         elif choice == "2":
27             print("\nДобавление нового фильма")
28             title = input("Название: ")
29             genre = input("Жанр: ")
30             duration = int(input("Длительность (мин): "))
31             age_rating = input("Возрастной рейтинг: ")
32             release_year = int(input("Год релиза: "))
33             description = input("Описание: ")
34             add_movie(title, genre, duration, age_rating, release_year, description)
```



```

34         add_movie(title, genre, duration, age_rating, release_year, description)
35         print("Фильм успешно добавлен.")
36
37     elif choice == "3":
38         print("\nДобавление нового сеанса")
39         try:
40             movie_id = int(input("ID фильма: "))
41             hall_id = int(input("ID зала: "))
42             start_time = input("Время начала (ГГГГ-ММ-ДД ЧЧ:ММ): ")
43             ticket_price = float(input("Цена билета: "))
44             format_type = input("Формат (2D, 3D, IMAX): ")
45             add_session(movie_id, hall_id, start_time, ticket_price, format_type)
46             print("Сеанс успешно добавлен.")
47         except ValueError:
48             print("Ошибка: проверьте ввод данных.")
49
50     elif choice == "4":
51         print("\nДобавление нового отдела")
52         name = input("Название отдела: ")
53         add_department(name)
54         print("Отдел успешно добавлен.")
55
56     elif choice == "5":
57         print("\nДобавление нового сотрудника")
58         full_name = input("ФИО: ")
59         try:
60             dept_id = int(input("ID отдела: "))
61             add_employee(full_name, dept_id)
62             print("Сотрудник успешно добавлен.")
63         except ValueError:
64             print("Ошибка: ID отдела должно быть числом.")
65
66     elif choice == "0":
67         print("Завершение работы. До свидания!")
68         sys.exit()
69
70     else:
71         print("Ошибка: введите число из списка.")
72
73 if __name__ == "__main__":
74     try:
75         main()
76     except KeyboardInterrupt:
77         print("\nПрограмма принудительно остановлена.")

```

src/reports.py

CodeBlame37 lines (31 loc) · 1.26 KB

RawCopyDownload

```
1      m.movie_id,
2          m.title,
3          m.genre,
4          m.duration,
5          m.age_rating,
6          m.release_year,
7          m.description,
8          d.name AS director_name
9  FROM movies m
10 LEFT JOIN directors d ON m.director_id = d.director_id
11 ORDER BY m.movie_id;
12 """
13
14 try:
15     with get_connection() as conn:
16         conn.row_factory = sqlite3.Row
17         cursor = conn.cursor()
18         cursor.execute(query)
19         rows = cursor.fetchall()
20
21     if not rows:
22         print("\n[] Список фильмов пуст.")
23         return
24
25     print("\n" + "=" * 120)
26     print(
27         f"{'ID':<3} | {'Название':<20} | {'Жанр':<10} | {'Длительность':<12} | {'Рейтинг':<8} | {'Год':<4} | {'Описание':<25} | {'Режиссер'}")
28     print("-" * 120)
29
30     for row in rows:
31         for row in rows:
32             print(
33                 f"{row['movie_id']:<3} | {row['title']:<20} | {row['genre']:<10} | {row['duration']:^12} | {row['age_rating']:<8} | {row['release_year']:<4} | {row[''
34             print("=" * 120)
35
36 except Exception as e:
37     print(f"Ошибка при формировании отчета: {e}")
```

src/services.py

Code

Blame

76 lines (71 loc) · 3.28 KB



```
1 import sqlite3
2 from db import get_connection
3
4  def add_employee(full_name, dept_id):
5     """
6     Добавляет нового сотрудника в таблицу employees.
7     :param full_name: Строка с именем сотрудника
8     :param dept_id: ID отдела (число)
9     """
10    query = "INSERT INTO employees (full_name, dept_id) VALUES (?, ?)"
11    try:
12        with get_connection() as conn:
13            cursor = conn.cursor()
14            cursor.execute(query, (full_name, dept_id))
15            conn.commit() # Сохраняем изменения
16            print(f"Успех: Сотрудник '{full_name}' добавлен в базу.")
17    except sqlite3.IntegrityError as e:
18        print(f"Ошибка: Не удалось добавить сотрудника. Проверьте ID отдела. ({e})")
19    except sqlite3.Error as e:
20        print(f"Произошла ошибка при работе с БД: {e}")
21
22  def add_department(name):
23      """Добавляет новый отдел (справочник)"""
24      query = "INSERT INTO departments (name) VALUES (?)"
25      try:
26          with get_connection() as conn:
27              conn.execute(query, (name,))
28              conn.commit()
29              print(f"Отдел '{name}' успешно создан.")
30      except sqlite3.Error as e:
31          print(f"Ошибка при создании отдела: {e}")
32
33  def add_movie(title, genre, duration, age_rating, release_year, description):
```

```

33  ✓ def add_movie(title, genre, duration, age_rating, release_year, description):
34      """
35      Добавляет новый фильм в таблицу movies.
36      :param title: Название фильма
37      :param genre: Жанр
38      :param duration: Длительность (мин)
39      :param age_rating: Рейтинг (например, "16+")
40      :param release_year: Год релиза
41      :param description: Описание
42      """
43      query = """
44      INSERT INTO movies (title, genre, duration, age_rating, release_year, description)
45      VALUES (?, ?, ?, ?, ?, ?)
46      """
47      try:
48          with get_connection() as conn:
49              conn.execute(query, (title, genre, duration, age_rating, release_year, description))
50              conn.commit()
51              print(f"Фильм '{title}' успешно добавлен.")
52      except sqlite3.Error as e:
53          print(f"Ошибка при добавлении фильма: {e}")
54
55  ✓ def add_session(movie_id, hall_id, start_time, ticket_price, format_type):
56      """
57      Добавляет новый сеанс.
58      :param movie_id: ID фильма
59      :param hall_id: ID зала
60      :param start_time: Время начала (строка)
61      :param ticket_price: Цена билета (float)
62      :param format_type: Формат (2D, 3D, IMAX)

```

```

57     Добавляет новый сеанс.
58     :param movie_id: ID фильма
59     :param hall_id: ID зала
60     :param start_time: Время начала (строка)
61     :param ticket_price: Цена билета (float)
62     :param format_type: Формат (2D, 3D, IMAX)
63     """
64     query = """
65     INSERT INTO sessions (movie_id, hall_id, start_time, ticket_price, format_type)
66     VALUES (?, ?, ?, ?, ?)
67     """
68     try:
69         with get_connection() as conn:
70             conn.execute(query, (movie_id, hall_id, start_time, ticket_price, format_type))
71             conn.commit()
72             print("Сеанс успешно добавлен.")
73     except sqlite3.Error as e:
74         print(f"Ошибка при добавлении сеанса: {e}")
75
76     # Здесь вы можете добавлять дополнительные функции для работы с другими таблицами

```

Вывод: Мы упорядочили директорию, заполнили её кодом и привели проект ближе к его завершающей форме!

1. **Архитектура и развертывание:** Использование `schema.sql` позволяет хранить структуру базы отдельно от кода. Это стандарт индустрии: вы запускаете один скрипт инициализации, и база готова к работе, что критично для командной разработки.
2. **Надежный CRUD:** Мы научились не просто «записывать данные», а управлять их жизненным циклом (Create, Read, Update, Delete). Использование **параметризованных запросов (?)** вместо `f`-строк защищает проект от SQL-инъекций.
3. **Бизнес-логика и отчетность:** Мы перешли от хранения строк к извлечению пользы. Написание SQL-запросов с агрегацией (`GROUP BY`, `SUM`, `JOIN`) позволяет формировать аналитические отчеты и реализовывать правила вашего проекта (например, проверку остатков на складе или расчет рейтинга пользователя).

4. **Командный стандарт:** Оформление кода через модули (например, выделение функций работы с БД в `database.py` или `models.py`) делает проект понятным для коллег. Использование относительных путей и чистого SQL позволяет любому члену команды запустить проект без сложной настройки окружения.